SLE-4: Final Viva & Architecture Decision Record (ADR)

Course: 02AML204 – Introduction to Artificial Intelligence

PRN	25UAM048
Name	Rajvardhan Pravin Patil
Division	A
Date	11 October 2026

PART A: Architecture Decision Record (ADR)

https://github.com/rajpatil31648/SLE2_IAI.git

https://github.com/rajpatil31648/SLE3_BFS_DFS_25UAM048.git

Title: Use of BFS and DFS with Separate Queue and Stack Structures

Status: Accepted

Date: 11 October 2026

PRN & Name: 25UAM048 – Rajvardhan Pravin Patil

1. Context

My project is a Python-based BFS vs DFS Search System that searches a graph from start node 0 to goal node 749. The purpose is to understand and compare two uninformed search algorithms on the same graph. BFS explores nodes level by level using a queue, while DFS explores along a path using a stack. In SLE-2, I profiled the algorithms and recorded the number of expanded nodes and execution time. In SLE-3, I documented the system using the C4 model, including its context, containers, components, and code-level overview.

2. Decision

I decided to keep BFS and DFS as separate search components in one comparison system, using a queue for BFS and a stack for DFS. Both algorithms use the same graph, start node 0, and goal node 749 so their results can be compared under the same setup.

3. Alternatives Considered

- Alternative 1: Implement only BFS. This would demonstrate level-by-level search but would not allow a direct comparison with DFS.
- Alternative 2: Implement only DFS. This would demonstrate stack-based traversal but would not provide the BFS comparison.
- Alternative 3: Use one shared traversal strategy for both. This could hide the key difference between the queue-based and stack-based search methods.

4. Rationale / Justification

- Keeping both algorithms makes the project useful for comparing search behaviour rather than demonstrating only one method.
- The SLE-2 results recorded 173 expanded nodes for BFS and 750 expanded nodes for DFS on the tested 750-node graph.
- The same start node, goal node, and graph were used for both algorithms, making the comparison consistent.
- The SLE-2 report noted that DFS had the lower measured execution time in that test; this is an observation for the tested graph, not a guarantee that DFS is always faster.
- BFS uses a queue to process the frontier in first-in, first-out order. DFS uses a stack to explore in last-in, first-out order.
- The SLE-3 Component diagram represents BFS and DFS separately and includes a visited set, goal test, and node counter.
- The Container diagram separates graph/configuration, search execution, performance measurement, and result output, making the system easier to understand and maintain.
- This design supports explaining both the algorithms and their measured behaviour during the viva.

5. Consequences

Positive:

- The two algorithms can be compared under the same conditions.
- Separate queue and stack structures make the algorithmic differences clear.
- The modular C4 design makes the system easier to explain, test, and extend.

Negative / Trade-offs:

- Maintaining two search implementations requires extra code and testing.
- The measured results depend on the graph structure and configuration; they may differ on another graph.

6. References

SLE-2: BFS vs DFS profiling results (as recorded in my SLE-3 document).

SLE-3: Architectural Design (Full C4 Model).

GitHub: https://github.com/rajpatil31648/SLE3_BFS_DFS_25UAM048.git

PART B: Short Summary of My SLE Journey

In SLE-1, I built the BFS/DFS search code and prepared the project documentation and AI contribution record. In SLE-2, I profiled BFS and DFS and compared their execution time and expanded-node counts. For the tested 750-node graph, the recorded counts were 173 expanded nodes for BFS and 750 for DFS, and DFS had the lower measured execution time. In SLE-3, I described the system using the C4 model: Context, Container, Component, and Code-level overview. In SLE-4, this ADR explains why I kept both algorithms and used separate queue and stack structures, while considering alternatives and trade-offs.

PART C: AI Contribution Note

AI tools used: ChatGPT.

What AI helped with: Organizing the ADR according to the SLE-4 guideline, improving clarity, and preparing short viva revision points.

What I did myself: The project is based on my BFS/DFS work. I reviewed the design, used my SLE-2 profiling results and SLE-3 architecture, and will verify that the final document matches my actual code and reports.

PART D: Viva Preparation Notes

- Why didyoukeep both BFSand DFS? Tocompare two search strategies on the same graph and configuration.
- What were your SLE-2 results? On the tested 750-node graph, BFS expanded 173 nodes and DFS expanded 750 nodes. DFS had the lower measured execution time in that test.
- Why does BFS use a queue and DFS use a stack? A queue processes the oldest discovered node first; a stack processes the most recently added node first.
- What does the C4 Container diagram show? It shows the graph/configuration module, search engine, performance measurement, and result/output module.
- Will these results always be the same? No. Performance depends on graph structure, neighbour order, and start/goal nodes.